Category C Problems - Solutions

Arrays, loops and conditionals only | Java | all samples verified by running the code

L1. Two Sum

Signature: `int[] twoSum(int[] nums, int target)`

Approach

• Two nested loops: the outer index i picks the first item, the inner index j starts at i + 1 so the same element is never used twice and each pair is checked only once.
• The first time nums[i] + nums[j] == target, return {i, j} immediately.
Complexity: Time O(n^2), Space O(1).

Solution

```
// L1. Two Sum
static int[] twoSum(int[] nums, int target) {
    for (int i = 0; i < nums.length; i++) {
        for (int j = i + 1; j < nums.length; j++) {
            if (nums[i] + nums[j] == target) {
                return new int[] { i, j };
            }
        }
    }
    return new int[] {}; // never reached: exactly one valid pair is guaranteed
}
```

Sample results

Input	Output
nums = [2, 7, 11, 15], target = 9	[0, 1]
nums = [3, 2, 4], target = 6	[1, 2]

L2. Best Time to Buy and Sell Stock

Signature: `int maxProfit(int[] prices)`

Approach

• Single left-to-right pass keeping two values: the lowest price seen so far (minPrice) and the best profit so far (maxProfit).
• At each day: update minPrice if today is cheaper, compute profit = today - minPrice, and keep it if it beats maxProfit. If prices only fall, maxProfit stays 0.
Complexity: Time O(n), Space O(1).

Solution

```
// L2. Best Time to Buy and Sell Stock
static int maxProfit(int[] prices) {
    if (prices.length == 0) return 0;
    int minPrice = prices[0];
    int maxProfit = 0;
    for (int i = 1; i < prices.length; i++) {
        if (prices[i] < minPrice) {
            minPrice = prices[i];
        }
        int profit = prices[i] - minPrice;
        if (profit > maxProfit) {
            maxProfit = profit;
        }
    }
    return maxProfit;
}
```

Sample results

Input	Output
prices = [7, 1, 5, 3, 6, 4]	5
prices = [7, 6, 4, 3, 1]	0

L3. Contains Duplicate

Signature: `boolean containsDuplicate(int[] nums)`

Approach

• Compare every element at i with every later element at j (j starts at i + 1, so positions always differ).
• Return true as soon as two equal values are found (early exit). If both loops finish with no match, return false.
Complexity: Time O(n^2), Space O(1).

Solution

```
// L3. Contains Duplicate
static boolean containsDuplicate(int[] nums) {
    for (int i = 0; i < nums.length; i++) {
        for (int j = i + 1; j < nums.length; j++) {
            if (nums[i] == nums[j]) {
                return true;
            }
        }
    }
    return false;
}
```

Sample results

Input	Output
nums = [1, 2, 3, 1]	true
nums = [1, 2, 3, 4]	false

L4. Merge Two Sorted Arrays

Signature: `int[] mergeSortedArrays(int[] arr1, int[] arr2)`

Approach

• Keep one pointer per array (i, j) and a write index k into a new result array of size arr1.length + arr2.length.
• While both arrays have elements left, copy the smaller current element and advance that pointer. When one array runs out, copy the remainder of the other. An empty array is handled naturally, because the first loop never runs.
Complexity: Time O(n + m), Space O(n + m).

Solution

```
// L4. Merge Two Sorted Arrays
static int[] mergeSortedArrays(int[] arr1, int[] arr2) {
    int[] result = new int[arr1.length + arr2.length];
    int i = 0, j = 0, k = 0;
    while (i < arr1.length && j < arr2.length) {
        if (arr1[i] <= arr2[j]) {
            result[k++] = arr1[i++];
        } else {
            result[k++] = arr2[j++];
        }
    }
    while (i < arr1.length) {
        result[k++] = arr1[i++];
    }
    while (j < arr2.length) {
        result[k++] = arr2[j++];
    }
    return result;
}
```

Sample results

Input	Output
arr1 = [1, 3, 5], arr2 = [2, 4, 6]	[1, 2, 3, 4, 5, 6]
arr1 = [], arr2 = [1, 2, 3]	[1, 2, 3]

L5. Rotate Array

Signature: `int[] rotateArray(int[] nums, int k)`

Approach

• First reduce k with k = k % n, since rotating by a multiple of the length changes nothing.
• Create a new array of the same size and place each element at its rotated position: rotated[(i + k) % n] = nums[i]. The modulo wraps indexes past the end back to the front.
Complexity: Time O(n), Space O(n).

Solution

```
// L5. Rotate Array
static int[] rotateArray(int[] nums, int k) {
    int n = nums.length;
    if (n == 0) return nums;
    k = k % n;
    int[] rotated = new int[n];
    for (int i = 0; i < n; i++) {
        rotated[(i + k) % n] = nums[i];
    }
    return rotated;
}
```

Sample results

Input	Output
nums = [1, 2, 3, 4, 5, 6, 7], k = 3	[5, 6, 7, 1, 2, 3, 4]
nums = [1, 2], k = 3	[2, 1] (k % 2 = 1)

Complete Program (Solutions.java)

```
import java.util.Arrays;

public class Solutions {

    // L1. Two Sum
    static int[] twoSum(int[] nums, int target) {
        for (int i = 0; i < nums.length; i++) {
            for (int j = i + 1; j < nums.length; j++) {
                if (nums[i] + nums[j] == target) {
                    return new int[] { i, j };
                }
            }
        }
        return new int[] {}; // never reached: exactly one valid pair is guaranteed
    }

    // L2. Best Time to Buy and Sell Stock
    static int maxProfit(int[] prices) {
        if (prices.length == 0) return 0;
        int minPrice = prices[0];
        int maxProfit = 0;
        for (int i = 1; i < prices.length; i++) {
            if (prices[i] < minPrice) {
                minPrice = prices[i];
            }
            int profit = prices[i] - minPrice;
            if (profit > maxProfit) {
                maxProfit = profit;
            }
        }
        return maxProfit;
    }

    // L3. Contains Duplicate
    static boolean containsDuplicate(int[] nums) {
        for (int i = 0; i < nums.length; i++) {
            for (int j = i + 1; j < nums.length; j++) {
                if (nums[i] == nums[j]) {
                    return true;
                }
            }
        }
        return false;
    }

    // L4. Merge Two Sorted Arrays
    static int[] mergeSortedArrays(int[] arr1, int[] arr2) {
        int[] result = new int[arr1.length + arr2.length];
        int i = 0, j = 0, k = 0;
        while (i < arr1.length && j < arr2.length) {
            if (arr1[i] <= arr2[j]) {
                result[k++] = arr1[i++];
            } else {
                result[k++] = arr2[j++];
            }
        }
        while (i < arr1.length) {
            result[k++] = arr1[i++];
        }
        while (j < arr2.length) {
            result[k++] = arr2[j++];
        }
        return result;
    }
    // L5. Rotate Array
```

```
    static int[] rotateArray(int[] nums, int k) {
        int n = nums.length;
        if (n == 0) return nums;
        k = k % n;
        int[] rotated = new int[n];
        for (int i = 0; i < n; i++) {
            rotated[(i + k) % n] = nums[i];
        }
        return rotated;
    }

    public static void main(String[] args) {
        System.out.println("L1: " + Arrays.toString(twoSum(new int[] {2, 7, 11, 15}, 9)));
        System.out.println("L1: " + Arrays.toString(twoSum(new int[] {3, 2, 4}, 6)));
        System.out.println("L2: " + maxProfit(new int[] {7, 1, 5, 3, 6, 4}));
        System.out.println("L2: " + maxProfit(new int[] {7, 6, 4, 3, 1}));
        System.out.println("L3: " + containsDuplicate(new int[] {1, 2, 3, 1}));
        System.out.println("L3: " + containsDuplicate(new int[] {1, 2, 3, 4}));
        System.out.println("L4: " + Arrays.toString(mergeSortedArrays(new int[] {1, 3, 5}, new int[] {2, 4
        System.out.println("L4: " + Arrays.toString(mergeSortedArrays(new int[] {}, new int[] {1, 2, 3})))
        System.out.println("L5: " + Arrays.toString(rotateArray(new int[] {1, 2, 3, 4, 5, 6, 7}, 3)));
        System.out.println("L5: " + Arrays.toString(rotateArray(new int[] {1, 2}, 3)));
    }
}
```